MANUAL DE ESTUDIO | EDICIÓN TÉCNICA

Documentación técnica del sistema de reservas

Modelos de datos, arquitectura, módulos, procesos, seguridad y operación explicados desde el código existente.

Fecha de revisión del código: 20 de octubre de 2026

Referencia Git: 88e0f20 | Aplicación Laravel monolítica

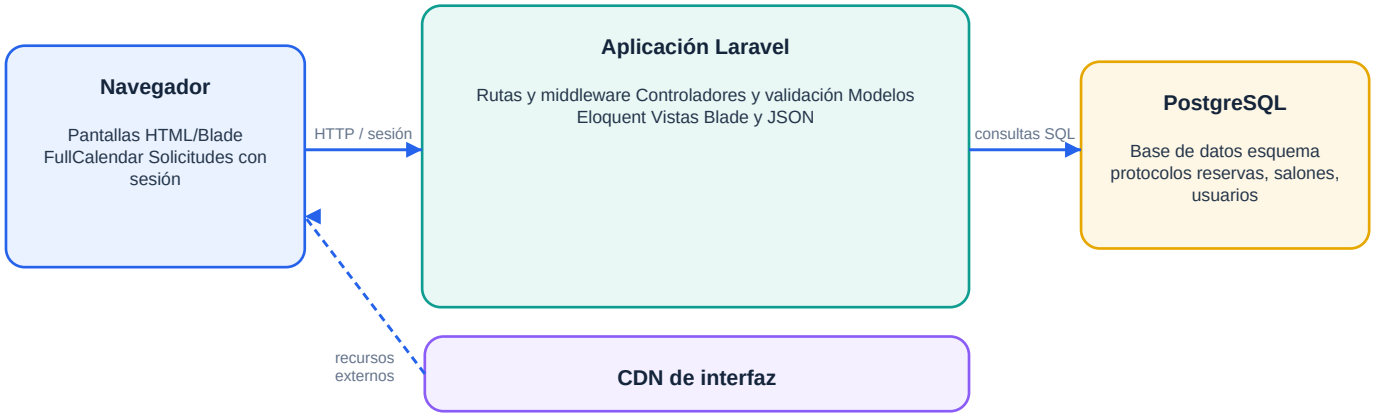
Cómo leer esta guía
Describe el código y las migraciones que se revisaron. Los puntos marcados como propuesta o pendiente no están configurados todavía. No incluye claves, contraseñas ni datos personales reales.

Alcance: los 15 documentos técnicos solicitados. Incluye diagramas originales para estudiar la arquitectura, el modelo de datos y el flujo de una reserva.

00 Resumen y estado de la documentación

La aplicación combina interfaz y servidor en un proyecto Laravel. Las responsabilidades están separadas por carpetas, pero el frontend aún no es un proyecto independiente.

El proyecto actual es monolítico: Laravel sirve la pantalla y procesa sus solicitudes.



N.º	Documento	Estado según repositorio
1	DER	Modelo conceptual inferido de migraciones; referencias entre salón y reserva no usan FK.
2	Diccionario de datos	Columnas principales documentadas desde migraciones; verificar contra la base instalada.
3-5	Arquitectura, módulos y procesos	Diagramas basados en rutas, controladores, modelos y vistas.
6	Inventario tecnológico	Versiones contrastadas en manifiestos y archivos lock; SO productivo pendiente.
7-8	Base de datos, API e integraciones	Objetos y rutas revisados estáticamente; no se consultó una base viva.
9-12	Instalación, roles, respaldos y seguridad	Instalación local y controles observados; operación productiva pendiente.
13	Plan de pruebas	Hay pruebas base de autenticación y perfil; no se ejecutaron en esta revisión.
14-15	Cambios e infraestructura	Historial Git incluido; servidores productivos y respaldos no están inventariados.

Advertencia de alcance
Este PDF documenta y revisa el código; no certifica seguridad, disponibilidad, integridad de una base en producción ni despliegue. Los datos marcados PENDIENTE deben confirmarse con el responsable técnico.

01 Diagrama Entidad-Relación (DER)

El esquema principal contiene usuarios, reservas y salones. Las relaciones punteadas son referencias lógicas guardadas como texto, no claves foráneas declaradas en las migraciones revisadas.

Las migraciones no declaran claves foráneas entre estas tablas.

Lectura del modelo

- SALONES.nombre se compara con RESERVAS.salon. El controlador también actualiza reservas activas y en papelera cuando renombra un salón.
- RESERVAS.registrado_por y RESERVAS.modificado_por guardan texto; no apuntan a USUARIOS.id.
- La tabla tokens_restablecimiento usa correo como clave primaria y relaciona la recuperación por correo sin una FK declarada.
- RESERVAS.eliminado_en aplica borrado lógico mediante SoftDeletes. La papelera mantiene registros para restaurar.

Punto para el estudio
Una relación visual no siempre es una restricción de la base. El sistema valida varios vínculos en Laravel, pero la base no impide por sí sola todas las referencias huérfanas.

02 Diccionario de datos

Tipos y restricciones se deducen de las migraciones de Laravel. VARCHAR usa longitud 255 si la migración usa string() sin longitud explícita; BIGINT corresponde a id().

Tabla RESERVAS

Columna	Tipo	Nulo	Clave / regla	Descripción
id	BIGINT	No	PK	Identificador de la reserva.
titulo	VARCHAR(255)	No		Título. El prefijo [CANCELADO] almacena estado.
fecha_inicio	TIMESTAMP	No		Inicio del evento.
fecha_fin	TIMESTAMP	No		Fin; se valida posterior al inicio.
salon	VARCHAR(255)	No	Lógica	Nombre del salón; coincide con salones.nombre.
nombre_usuario	VARCHAR(255)	Sí		Nombre de quien solicita.
telefono_usuario	VARCHAR(255)	Sí		Teléfono del solicitante.
nombre_autoridad	VARCHAR(255)	Sí		Autoridad asociada.
responsable	VARCHAR(255)	Sí		Responsable del evento.
tipo_evento	VARCHAR(255)	Sí		Categoría del evento.
requerimientos	JSON	Sí		Lista estructurada de requerimientos.
aforo	INTEGER	Sí		Cantidad estimada de personas.
tipo_ingreso	VARCHAR(255)	Sí		Modalidad de ingreso.
nombre_coordinador_externo	VARCHAR(255)	Sí		Coordinador externo.
telefono_coordinador_externo	VARCHAR(255)	Sí		Teléfono del coordinador externo.
requerimientos_especiales	TEXT	Sí		Detalle largo de necesidades.
registrado_por	VARCHAR(255)	Sí		Nombre o correo que se mostró al registrar.
coordinador_interno	VARCHAR(255)	Sí		Coordinador interno.
modificado_por	VARCHAR(255)	Sí	Auditoría	Actor del último cambio; no hay historial por cambio.
fecha_creacion	TIMESTAMP	Sí		Fecha de creación.
fecha_actualizacion	TIMESTAMP	Sí		Fecha de última modificación.
eliminado_en	TIMESTAMP	Sí	Soft delete	NULL indica activo; fecha indica papelera.

02 Diccionario de datos (continuación)

Columnas observadas en las migraciones del catálogo y autenticación.

Tabla SALONES

Columna	Tipo	Nulo	Clave / regla	Descripción
id	BIGINT	No	PK	Identificador del salón.
nombre	VARCHAR(255)	No	UNIQUE	Nombre mostrado; único en la tabla.
color	VARCHAR(7)	No	Validación de app	Color hexadecimal. La app evita repetirlo y no ofrece edición.
activo	BOOLEAN	No	DEFAULT TRUE	Disponibilidad para nuevas reservas.
orden	INTEGER	No	DEFAULT 0	Orden en filtros y calendario.
fecha_creacion	TIMESTAMP	Sí		Fecha de creación.
fecha_actualizacion	TIMESTAMP	Sí		Fecha de actualización.

Tablas de USUARIOS y TOKENS

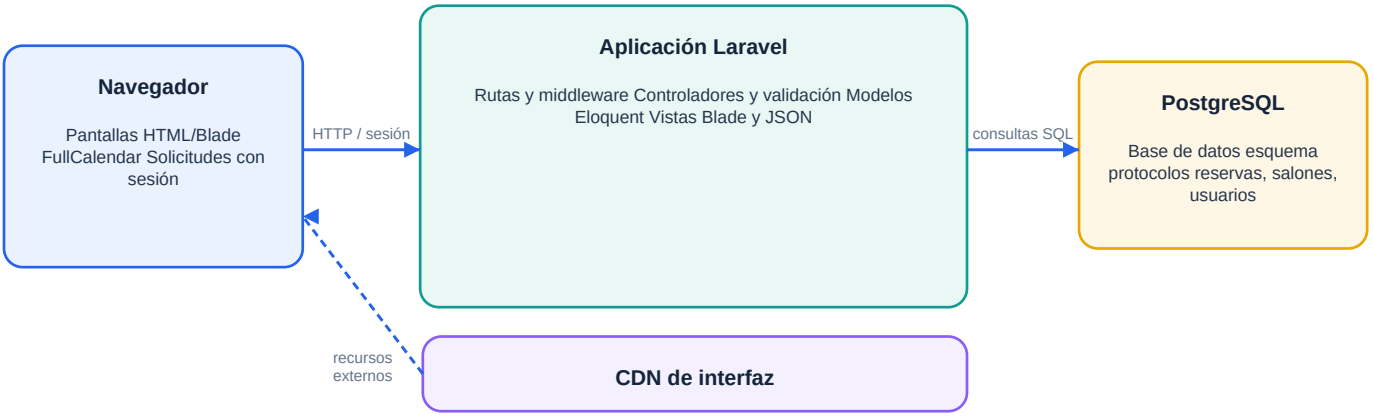
Tabla.columna	Tipo	Nulo	Clave / valor	Descripción
usuarios.id	BIGINT	No	PK	Identificador.
usuarios.nombre	VARCHAR(255)	No		Nombre visible.
usuarios.correo	VARCHAR(255)	No	UNIQUE	Correo para acceso y notificaciones.
usuarios.telefono	VARCHAR(255)	Sí		Teléfono de contacto.
usuarios.rol	VARCHAR(255)	No	DEFAULT funcionario	Rol inicial de la migración. El registro web asigna usuario.
usuarios.correo_verificado_en	TIMESTAMP	Sí		Marca de verificación.
usuarios.contrasena	VARCHAR(255)	No	Hash	Hash, no texto legible.
usuarios.token_recordar	VARCHAR(100)	Sí		Token de sesión persistente.
usuarios.fecha_creacion / fecha_actualizacion	TIMESTAMP	Sí		Fechas de auditoría básicas.
tokens_restablecimiento.correo	VARCHAR(255)	No	PK	Correo objetivo de recuperación.
tokens_restablecimiento.token	VARCHAR(255)	No	Hash	Token de recuperación cifrado/hash.
tokens_restablecimiento.fecha_creacion	TIMESTAMP	Sí		Fecha de emisión para controlar vigencia.

Límite de verificación
El documento refleja las migraciones del repositorio, no una extracción directa del servidor. Antes de una migración real, comparar columnas y tipos con la base destino.

03 Diagrama de arquitectura del sistema

La interfaz y la lógica se ejecutan como un proyecto Laravel. PostgreSQL está configurado como motor de datos en config/database.php.

El proyecto actual es monolítico: Laravel sirve la pantalla y procesa sus solicitudes.



Componentes y comunicación

Componente	Responsabilidad	Ubicación
Navegador	Presenta pantallas y envía solicitudes autenticadas.	Cliente web
Laravel	Aplica rutas, sesión, permisos, validación y lógica de negocio.	app/, routes/, bootstrap/
Blade / Vite	Renderiza HTML y prepara CSS/JavaScript local.	resources/views, resources/css, resources/js
PostgreSQL	Persiste usuarios, reservas y catálogo.	Conexión pgsql; schema configurado por DB_SCHEMA
CDN	Entrega FullCalendar, iconos y avisos al navegador.	jsDelivr y cdnjs usados desde vistas

Separación futura
El frontend aún depende de Blade, sesión Laravel y protección CSRF. Publicarlo por separado requiere diseñar autenticación, cookies, CORS y conectividad del backend antes de mover archivos.

04 Diagrama de módulos

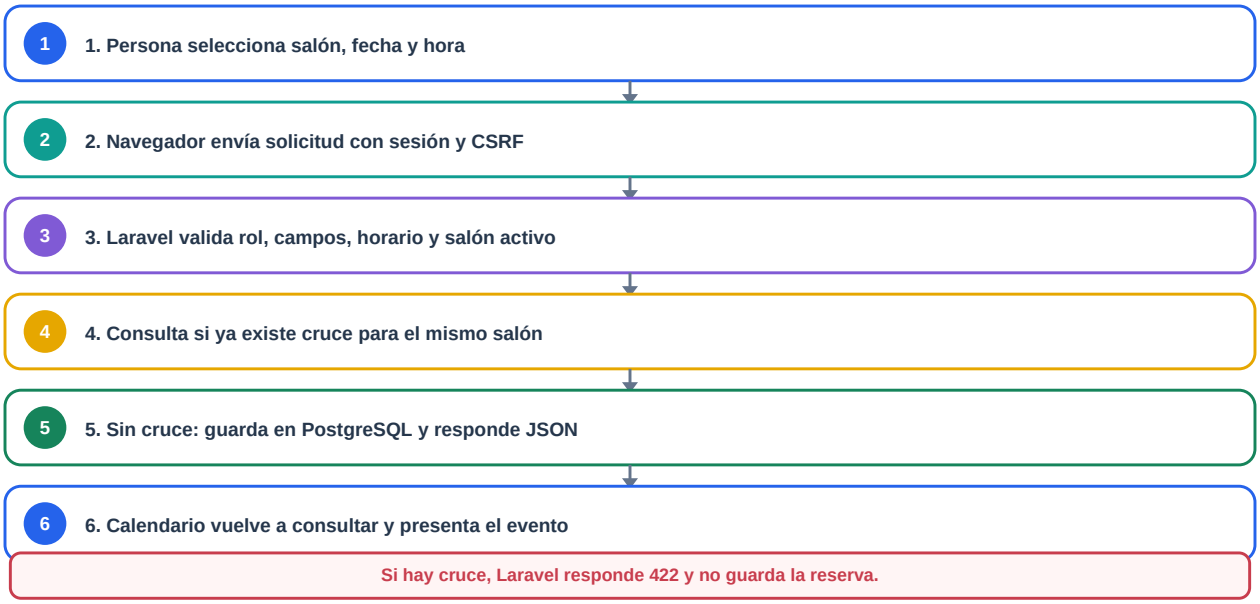
La vista principal está organizada por función; las rutas conectan cada módulo con sus controladores.

Módulo	Archivo servidor	Pantalla / respuesta
Calendario y disponibilidad	SalonController, ReservaController	salones/usar.blade.php y JSON de eventos
Supervisión y exportación	ReservaController	salones/supervisar.blade.php, CSV
Catálogo y cuentas	SalonController, UsuarioController	salones/usuarios.blade.php
Papelera e historial	ReservaController	salones/papelera.blade.php y supervisar_reservas.blade.php
Protocolo	ReservaController	protocolo.blade.php y partials/form_protocolo.blade.php
Autenticación y perfil	Auth controllers, ProfileController	auth/, profile/

Carpetas transversales: app/Models describe datos; app/Http/Requests valida formularios; app/Providers conecta adaptadores; config/ guarda ajustes; database/migrations versiona el esquema.

05 Diagrama de flujo de creación de una reserva

El flujo muestra la ruta habitual del calendario. Los controladores vuelven a validar datos aunque la interfaz ya haya realizado validaciones.



Flujos relacionados

Proceso	Camino resumido
Inicio de sesión	POST /login -> LoginRequest valida y limita intentos -> Laravel inicia sesión -> redirección al dashboard.
Editar / cancelar	Calendario selecciona evento -> PUT /api/salones/reservas/{id} -> valida rol y campos -> estado sólo superadmin -> actualiza título y actor.
Papelera	Admin solicita DELETE -> SoftDeletes llena eliminado_en -> admin puede restaurar o vaciar permanentemente.
Disponible	Mini calendario/campo define fecha -> tabla pide reservas -> calcula huecos por salón -> muestra hasta 10 filas.

06 Inventario tecnológico

Versiones contrastadas en composer.lock y package-lock.json al revisar el repositorio el 20/10/2026. Los valores de CDN se leen de las etiquetas script de las vistas.

Área	Tecnología	Versión declarada / bloqueada	Uso
Backend	PHP	Requisito ^8.2	Lenguaje del servidor.
Backend	Laravel Framework	12.69.3 bloqueado; ^12.0 en composer.json	Rutas, sesión, ORM, validación.
Backend	Laravel Breeze	2.4.2	Plantillas de autenticación base.
Backend	PostgreSQL	Motor pgsql; versión del servidor pendiente	Base de datos.
Frontend build	Node.js	No fijado en el repositorio	Ejecuta npm y Vite; registrar versión del equipo.
Frontend build	Vite	7.3.6 bloqueado; ^7.0.7 declarado	Compila CSS y JavaScript.
Frontend	Tailwind CSS	3.4.19 bloqueado; ^3.1.0 declarado	Utilidades CSS.
Frontend	Alpine.js / Axios	3.17.4 / 1.20.0 bloqueados	Interacciones y solicitudes HTTP.
Frontend CDN	FullCalendar	6.1.10	Calendario de día, semana y mes.
Frontend CDN	Font Awesome / SweetAlert2	6.4.0 / @11	Iconos y avisos emergentes.
Pruebas/formato	PHPUnit / Pint	11.5.56 / 1.30.4	Pruebas PHP y formato.
Plataforma	Windows local observado	Versión productiva no definida	Despliegue Nginx/Apache/PHP-FPM no está configurado en repo.

Composer administra paquetes PHP desde composer.json / composer.lock. npm administra paquetes del frontend desde package.json / package-lock.json. No editar vendor/ ni node_modules/. Los manifiestos JSON no aceptan comentarios.

07 Documentación técnica de la base de datos

La conexión de ejemplo es PostgreSQL, esquema protocolos. Las migraciones declaran las estructuras centrales y comentarios descriptivos para PostgreSQL.

Objeto	Definido / uso observado	Estado
Tablas de negocio	usuarios, reservas, salones, tokens_restablecimiento	Definidas en migraciones.
Historial de migraciones	historial_migraciones con id_migracion, nombre_migracion, lote	Repositorio Laravel adaptado a nombres españoles.
Claves e índices	PK en id/correo; UNIQUE en usuarios.correo y salones.nombre	Sin índices manuales ni FK en migraciones revisadas.
Borrado lógico	reservas.eliminado_en + SoftDeletes	Papelera de reservas.
Tabla eventos	Comentada como histórica; agenda actual consulta reservas	Estructura no creada por migraciones actuales; verificar base existente.
Cache y colas	Configuración de ejemplo usa archivo/sync	Migraciones vacías desactivan tablas DB de cache y colas.
Vistas SQL / funciones / procedimientos / triggers	No se encontraron definiciones en migraciones versionadas	Verificar objetos reales en PostgreSQL antes de producción.

Puntos de integridad

- Los conflictos de horario se comprueban en ReservaController, antes de guardar. No existe una exclusión de solapamientos declarada en PostgreSQL.
- El color único y la protección de bajas del catálogo se validan en Laravel; la migración sólo fija UNIQUE para el nombre.
- La tabla reservas.salon no tiene FK a salones. Los nombres deben conservarse sincronizados por reglas de la aplicación.

Incidente previo que explica el adaptador
El repositorio de migraciones traduce nombres internos migration/batch a nombre_migracion/lote. Si una base tiene historial_migraciones con otra estructura, artisan migrate falla; revisar el esquema real antes de ejecutar migraciones.

08 API e integraciones

No existe routes/api.php para estos endpoints: se declaran en routes/web.php dentro del middleware auth. Usan sesión Laravel y protección CSRF web para cambios.

Método y ruta	Entrada principal	Respuesta / acceso
GET /api/salones/reservas	Sin filtro obligatorio; eventos activos	JSON de eventos para FullCalendar; sesión requerida.
GET /api/salones/reservas/{id}	id en URL	Reserva JSON; 404 si no existe.
POST /api/salones/reservas	titulo, fecha_inicio, fecha_fin, salon; datos de contacto opcionales	200 {success:true}; 403 rol; 422 validación o cruce.
PUT /api/salones/reservas/{id}	Campos editables; estado opcional	200 éxito; sólo superadmin puede enviar estado; 403/404/422 según caso.
DELETE /api/salones/reservas/{id}	id en URL	Admin; soft delete; JSON {success:true}; 403 si otro rol.
GET /api/reservas/fechas	month, year, salon opcional	JSON de fechas ocupadas del mini calendario.

Integraciones externas

Integración	Uso	Estado
jsDelivr	FullCalendar 6.1.10 y locale español; SweetAlert2 @11	CDN desde el navegador.
cdnjs	Font Awesome 6.4.0	CDN desde el navegador.
Correo	MAIL_MAILER=log en .env.example	No hay proveedor de correo productivo confirmado.
AWS / pagos / otros servicios	Variables de muestra vacías; no aparece integración funcional revisada	Pendiente confirmar si existen fuera del repositorio.

09 Instalación y despliegue

Los pasos siguientes son para desarrollo local. Producción requiere decidir servidor, dominio, HTTPS, política de respaldo y método de despliegue.

Requisitos comprobables

- PHP compatible con ^8.2 y extensiones requeridas por Laravel.
- Composer para dependencias PHP.
- Node.js y npm para compilar recursos; versión instalada aún debe fijarse en un manual operativo.
- PostgreSQL accesible; crear base y esquema según decisión de instalación.
- Acceso a Internet para instalar paquetes y, en el navegador, descargar recursos CDN.

Desarrollo local

```
composer install
Copy-Item .env.example .env
php artisan key:generate
npm install
php artisan migrate
npm run dev # en otra terminal: php artisan serve
```

Antes de migrar, verificar DB_DATABASE, DB_SCHEMA, DB_HOST y el historial de migraciones. Nunca usar migrate:fresh en una base con datos.

Puesta en producción propuesta

Paso	Control
Preparar servidor	PHP-FPM con Nginx o Apache; raíz pública debe ser public/. El servidor productivo aún no está definido.
Proteger configuración	APP_ENV=production, APP_DEBUG=false, APP_URL HTTPS y secretos sólo en entorno del servidor.
Compilar	composer install --no-dev --optimize-autoloader; npm ci; npm run build.
Base de datos	Crear usuario PostgreSQL con privilegios mínimos; comprobar esquema y respaldo antes de migrate --force.
Caché / acceso	config:cache, route:cache, view:cache; HTTPS, firewall, CORS cerrado si se separa frontend.
Validar salida	Probar login, crear/editar, colisiones, roles, papelera, protocolo, exportación y recuperación.

10 Matriz de roles y permisos

Matriz basada en las comprobaciones que aparecen en controladores y rutas al 20/10/2026. Se distingue permiso explícito de permiso implícito por sólo exigir login.

Acción	usuario / funcionario	admin	superadmin	Control actual
Abrir rutas protegidas	Sí	Sí	Sí	Middleware auth; algunas rutas sólo requieren sesión.
Crear / editar reserva calendario	No en API	Sí	Sí	ReservaController verifica admin/superadmin.
Cambiar estado cancelado	No	No	Sí	Sólo superadmin cuando se envía estado.
Eliminar reserva / vaciar papelera	No	Sí	No	Métodos exigen exactamente admin.
Ver papelera	No	Sí	No	Método exige admin.
Restaurar reserva	usuario: no	Sí	Sí	Bloquea usuario/vacío; otros roles no estándar podrían pasar.
Administrar nombres/color/estado salones	No	Sí	Sí	Validador permite ambos roles; no sólo superadmin.
Crear/editar/eliminar cuentas	No	Sí	No	UsuarioController verifica admin exactamente.
Ver usuarios/supervisión/exportar	Revisar	Sí	Sí	Varias acciones no tienen guardia de rol en controlador.
Eliminar salón del catálogo	Revisar	Revisar	Revisar	destroy() no verifica rol ni reservas antes del delete físico.

Acción recomendada antes de producción
Alinear las reglas con la matriz aprobada: proteger SalonController.destroy, revisar acceso a listas de usuarios/supervisión/exportación y al formulario web de creación. Confirmar además si admin o sólo superadmin administra el catálogo. No se modificó código como parte de este documento.

Nota de roles: la migración da rol inicial funcionario, pero el registro web asigna usuario. Revisar cuentas existentes y normalizar nombres de rol antes de confiar en reglas por texto.

11 Plan de copias de seguridad y recuperación

No se encontró una tarea automática de respaldo versionada. Lo siguiente es una propuesta, no una operación activa.

Elemento	Propuesta inicial	Estado actual
Respaldo	pg_dump en formato custom, diario antes de ventana de cambios.	No configurado en el repositorio.
Retención	14 diarios y 12 mensuales; ajustar a política institucional.	Responsable y retención pendientes.
Ubicación	Copia cifrada fuera del servidor de base de datos, acceso restringido.	Servidor/almacenamiento productivo pendiente.
Verificación	Restauración mensual aislada; simulacro trimestral documentado.	Sin resultados de recuperación registrados.
Responsable	Administrador de infraestructura designado por la institución.	Nombre y suplente pendientes.

Ejemplo para crear respaldo

```
pg_dump --format=custom --file=reservas_YYYYMMDD.dump --dbname=sistema_certificados
```

Ejemplo conceptual de restauración en una base de pruebas vacía. No ejecutar contra producción sin autorización y plan de recuperación aprobado:

```
pg_restore --clean --if-exists --dbname=sistema_eventos_restore reservas_YYYYMMDD.dump
```

Datos de acceso
No guardar contraseñas en el PDF, scripts o historial de terminal. Usar el gestor institucional de secretos o el mecanismo seguro aprobado para el servidor.

12 Documentación de seguridad

Resumen de controles observados en código. No representa una prueba de penetración ni una auditoría completa de dependencias.

Área	Control observado	Pendiente / nota
Sesión	Rutas agrupadas con auth; dashboard requiere verificación de correo.	Confirmar expiración, cookies seguras y HTTPS en despliegue.
Contraseñas	Modelo usa cast hashed; controladores usan Hash::make.	Política de longitud y recuperación debe revisarse con la institución.
Fuerza bruta	LoginRequest bloquea tras cinco intentos por correo e IP.	Verificar límite y monitoreo en entorno productivo.
Validación	Controladores validan fechas, salón, campos y conflictos.	Completar pruebas de rol y solicitudes alternativas.
CSRF	Endpoints web usan sesión Laravel y middleware web.	No publicar como API independiente sin diseñar autenticación/CORS.
Auditoría	modificado_por almacena actor de último cambio.	No hay historial append-only de cada modificación.
Secretos	.env está ignorado por Git; .env.example contiene valores de muestra.	Rotar cualquier secreto compartido por canales no seguros.
Correo	Ejemplo usa mailer log.	No hay transportador de correo productivo confirmado.

Observaciones estáticas de permisos

- SalonController.destroy ejecuta eliminación física de un salón sin validar rol ni comprobar reservas asociadas.
- UsuarioController.index carga todas las cuentas y no revisa rol en el método; la ruta exige sesión, no rol.
- ReservaController.supervisar, supervisarReservas y exportarCsv no muestran una verificación de rol en el método revisado.
- guardarReservaCompleta no contiene el mismo control explícito admin/superadmin que el endpoint JSON del calendario.
- La autorización del catálogo permite admin y superadmin, a pesar de que el método se llama validarSuperadmin.

Prioridad
Resolver y probar los controles de autorización y eliminación física antes de exponer el sistema fuera de la red local. La interfaz no reemplaza controles del servidor.

13 Plan de pruebas técnicas

Se revisó el inventario de tests, pero no se ejecutó PHPUnit para este PDF. La columna Resultado queda pendiente de ejecución y evidencia.

ID	Caso sugerido	Resultado esperado	Resultado
T-01	Ingresar válido e inválido; bloqueo tras 5 intentos.	Sesión válida o error controlado / throttle.	Pendiente
T-02	Crear evento con intervalo válido.	JSON de éxito y fila persistida.	Pendiente
T-03	Crear evento con fin antes del inicio.	Validación 422; no se guarda.	Pendiente
T-04	Intentar cruce horario en el mismo salón.	Respuesta 422; no hay duplicidad.	Pendiente
T-05	Repetir horario en salones diferentes.	Se permite si reglas lo autorizan.	Pendiente
T-06	Cambiar a cancelado como superadmin y como admin.	Superadmin permite; admin recibe 403.	Pendiente
T-07	Eliminar, restaurar y vaciar papelera con roles.	Sólo roles autorizados; datos coherentes.	Pendiente
T-08	Mini calendario cambia fecha en Disponible.	Una fecha seleccionada y máximo 10 filas.	Pendiente
T-09	Ancho de pantalla móvil, día/semana/mes.	Controles utilizables y datos accesibles.	Pendiente
T-10	Respaldar y restaurar en base aislada.	Recuperación íntegra documentada.	Pendiente

Cobertura existente

Hay pruebas base de autenticación, contraseñas, verificación y perfil en tests/Feature. No se encontraron suites específicas para reservas, catálogo de salones, roles de administración, papelera ni API del calendario.

14 Historial de versiones y cambios

Resumen de los últimos commits disponibles al revisar Git. No se identificaron etiquetas de versión semántica en esta revisión.

Commit	Fecha	Asunto
88e0f20	20/10/2026	Actualización del proyecto
7427b99	19/10/2026	Actualicé la vista de disponibles
3d914de	19/10/2026	Merge pull request #2 desde versión local
24b7a9a	19/10/2026	Descripción de las actualizaciones realizadas
9e6b047	16/10/2026	Actualicé la exportación de secciones
8dcc8fb	16/10/2026	Merge pull request #1 desde versión local
362b73b	16/10/2026	Actualización del proyecto
311e56e	16/10/2026	Actualización del proyecto

Estado del árbol de trabajo
La revisión encontró archivos modificados y documentación local sin confirmar en Git. El hash 88e0f20 identifica HEAD, no incluye por sí solo todos los cambios locales visibles al generar este PDF.

Para cada cambio futuro, registrar fecha, persona responsable, motivo, archivos afectados y pruebas ejecutadas. Crear una etiqueta de versión sólo cuando exista un proceso de publicación acordado.

15 Inventario de infraestructura

Se separan valores locales de los datos productivos. Las direcciones privadas, credenciales y certificados deben guardarse en un inventario restringido, no en este PDF.

Elemento	Desarrollo observado / ejemplo	Producción
URL navegador	http://127.0.0.1:8007	Dominio y URL HTTPS: PENDIENTE
Laravel	php artisan serve en desarrollo	Servidor web/PHP-FPM: PENDIENTE
PostgreSQL	DB_HOST 127.0.0.1, DB_PORT 5432 en .env.example	Host, puerto y HA: PENDIENTE
Base / esquema	sistema_certificados / protocolos en ejemplo	Nombre real y permisos: PENDIENTE
Frontend assets	Vite local o public/build	CDN permitido / assets locales: PENDIENTE
TLS / certificado	No configurado en el repositorio	Emisor, vencimiento y renovación: PENDIENTE
Firewall / IPs privadas	No inventariado	Restringir a documento de acceso controlado.
Respaldos / monitoreo	Sin automatización documentada	Destino, alertas y responsables: PENDIENTE

Registro restringido complementario

En el repositorio institucional restringido se debe guardar: responsable y contacto de guardia, IP interna, hostname, sistema operativo y parcheado, puertos expuestos, firewall, certificados, cuentas técnicas, bóveda de secretos, retención de respaldos, proveedor y fecha de renovación. No incluir contraseñas ni llaves privadas en la copia general de estudio.

16 Ruta de estudio y glosario

Sugerencia para aprender el sistema desde una tarea concreta hasta el almacenamiento.

Orden	Lee / observa	Pregunta de aprendizaje
1	routes/web.php	¿Qué URL llama a qué método?
2	app/Http/Controllers/ReservaController.php	¿Qué valida el servidor antes de responder?
3	app/Models/Reserva.php y migraciones	¿Qué columnas se guardan y cómo se convierten fechas/JSON?
4	resources/views/salones/usar.blade.php	¿Qué HTML, estilos y eventos ve la persona?
5	DevTools del navegador / Network	¿Qué endpoint responde y qué JSON entrega?
6	tests/Feature	¿Qué comportamiento se comprueba automáticamente?
7	Git log y diff	¿Qué archivos y motivo corresponden a la versión?

Glosario corto

Término	Significado
Ruta	Regla que conecta una URL y método HTTP con una acción.
Controlador	Código del backend que valida, consulta y responde.
Modelo	Clase Laravel que representa una tabla.
Migración	Archivo versionado que crea o cambia el esquema de datos.
Blade	Plantilla Laravel que mezcla HTML con datos y directivas.
Soft delete	Marca de eliminación lógica que conserva la fila.
CSRF	Protección contra solicitudes web falsificadas usando sesión.
CDN	Servidor externo que entrega una librería al navegador.

Cambios de esta documentación
20/10/2026: se preparó este manual a partir de rutas, controladores, modelos, vistas, migraciones, manifiestos de dependencias, archivos lock, pruebas existentes y Git. Se añadieron diagramas de arquitectura, módulos, DER y flujo de creación.